

MovieLens Project: Predicting Movie Ratings

Juan Luis Rojas Contreras

July 2026

Table of Contents

1 1. Introduction

1.1 1.1 Project motivation

Recommendation systems are present on almost every digital platform we use every day. Streaming services give us suggestions through algorithms that try to predict what we might like based on our behavior and the behavior of other users with similar tastes.

In this project, the goal is to develop a recommendation system that can predict how a user would rate a movie they have not watched yet. For this purpose, we use the MovieLens 10M dataset, which contains millions of real user ratings.

1.2 1.2 Main objective

The objective is to build a model that can predict ratings with an RMSE lower than 0.86490, which is the threshold established to obtain the maximum score in this project.

2 2. Methods

2.1 2.1 The data

The dataset comes from GroupLens Research and contains approximately 10 million ratings. Each record includes:

- The user ID.
- The movie ID.
- The rating (from 0.5 to 5.0).
- Additional information such as the title and genres.

2.2 2.2 Data splitting

To evaluate our model correctly, we divided the data into three sets:

- **Training set (edx)**: 90% of the data, used to develop the model.
- **Final test set (final_holdout_test)**: 10% of the data, used only at the end to evaluate the model.

In addition, within the training data we created:

- **train_set**: 80% for training the models.
- **validation_set**: 20% for validating and tuning hyperparameters.

This split allows us to evaluate the model on data it never saw during training, which is more realistic.

2.3 2.3 Modeling approach

The approach we follow is **collaborative filtering**. The main idea is to predict a user's rating for a movie based on two main factors:

1. **Movie effect**: Some movies are better and tend to receive higher ratings.
2. **User effect**: Some users are more generous or more strict when rating.

The simplest model we use is the **global average**, which simply predicts the average of all ratings for any movie and user.

Then we add the **movie effect**, which adjusts the prediction according to how good or bad each movie is.

After that, we include the **user effect**, which accounts for whether a user tends to rate higher or lower than average.

Finally, we apply **regularization** to prevent the model from fitting the training data too closely.

2.4 2.4 Regularization

Regularization is a technique that removes extreme effects. For example, if a movie has very few ratings, its estimated effect could be very large or very small just by chance. Regularization shrinks those values toward zero to make the model more accurate.

The regularization parameter, λ , controls how much the effects are reduced:

- **Small λ** : little shrinkage, the effects fit the data more closely.
- **Large λ** : strong shrinkage, the effects move closer to zero.

We found the optimal value by trying different values and choosing the one with the lowest RMSE on the validation set.

3 Results

3.1 Data dimensions

```
print(paste("Training set:", nrow(train_set), "rows"))
## [1] "Training set: 7200089 rows"

print(paste("Validation set:", nrow(validation_set), "rows"))
## [1] "Validation set: 1799966 rows"

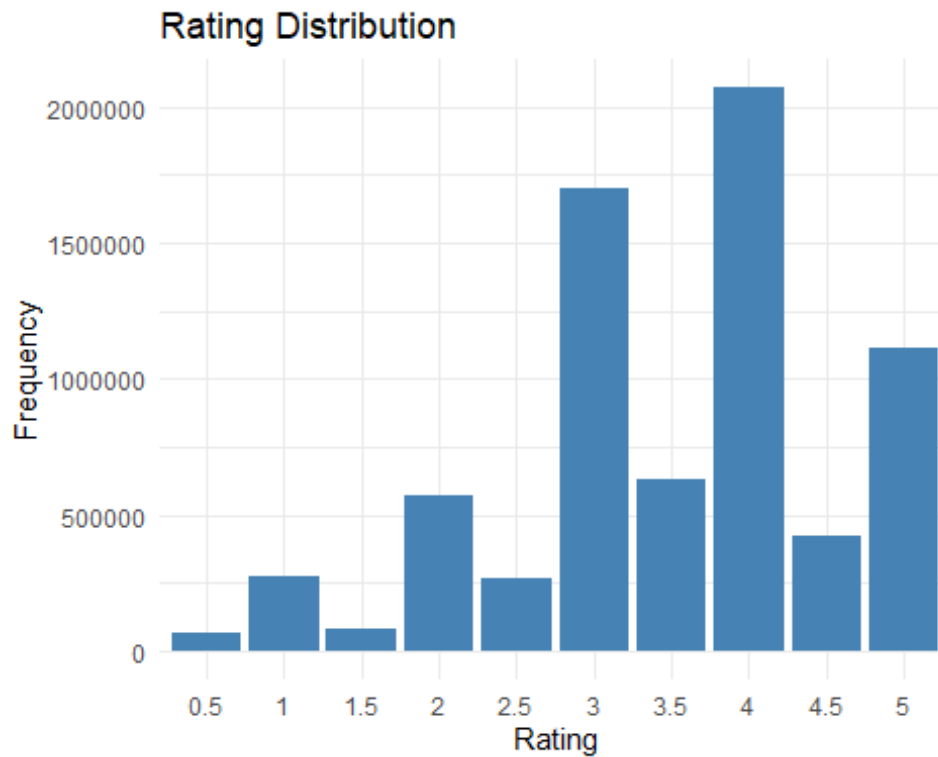
print(paste("Final test set:", nrow(final_holdout_test), "rows"))
## [1] "Final test set: 999999 rows"

print(paste("Unique movies:", length(unique(train_set$movieId))))
## [1] "Unique movies: 10677"

print(paste("Unique users:", length(unique(train_set$userId))))
## [1] "Unique users: 69878"
```

3.2 Rating distribution

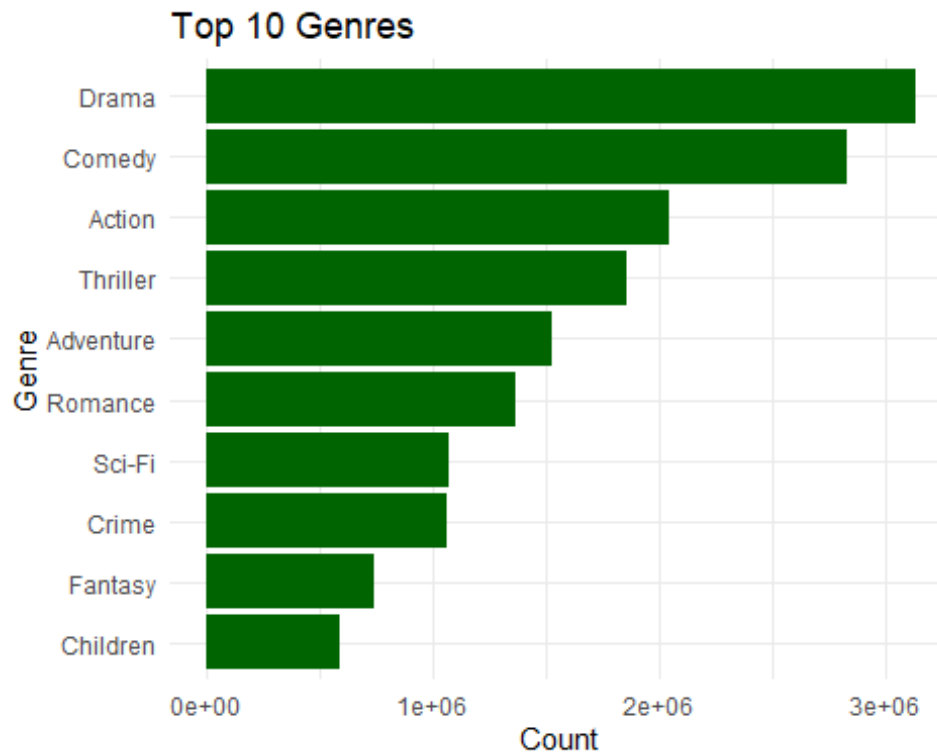
```
train_set %>%
  count(rating) %>%
  ggplot(aes(x = factor(rating), y = n)) +
  geom_col(fill = "steelblue") +
  labs(
    title = "Rating Distribution",
    x = "Rating",
    y = "Frequency"
  ) +
  theme_minimal()
```



It can be seen that users tend to be more positive, giving higher ratings than lower ones. As shown in the image, ratings 3 and 4 are the most common. Because of this behavior, the global average is a good starting point.

3.3 3.3 Top genres

```
train_set %>%
  separate_rows(genres, sep = "\\|") %>%
  count(genres, sort = TRUE) %>%
  head(10) %>%
  ggplot(aes(x = reorder(genres, n), y = n)) +
  geom_col(fill = "darkgreen") +
  coord_flip() +
  labs(
    title = "Top 10 Genres",
    x = "Genre",
    y = "Count"
  ) +
  theme_minimal()
```



Drama, Comedy, and Action are the genres with the most ratings in the dataset, but that does not mean they are the best rated. Since they have more observations, the model can estimate their associated effects more precisely.

3.4 Model performance

```
rmse_results %>%
  knitr::kable(
    caption = "Model Comparison",
    booktabs = TRUE,
    format.args = list(digits = 5)
  )
```

Model Comparison

method	RMSE
Global mean model	1.05990
Movie effect model	0.94374
Movie + user effect model	0.86593
Regularized movie + user effect model	0.86524

As shown in the table, each model improves on the previous one:

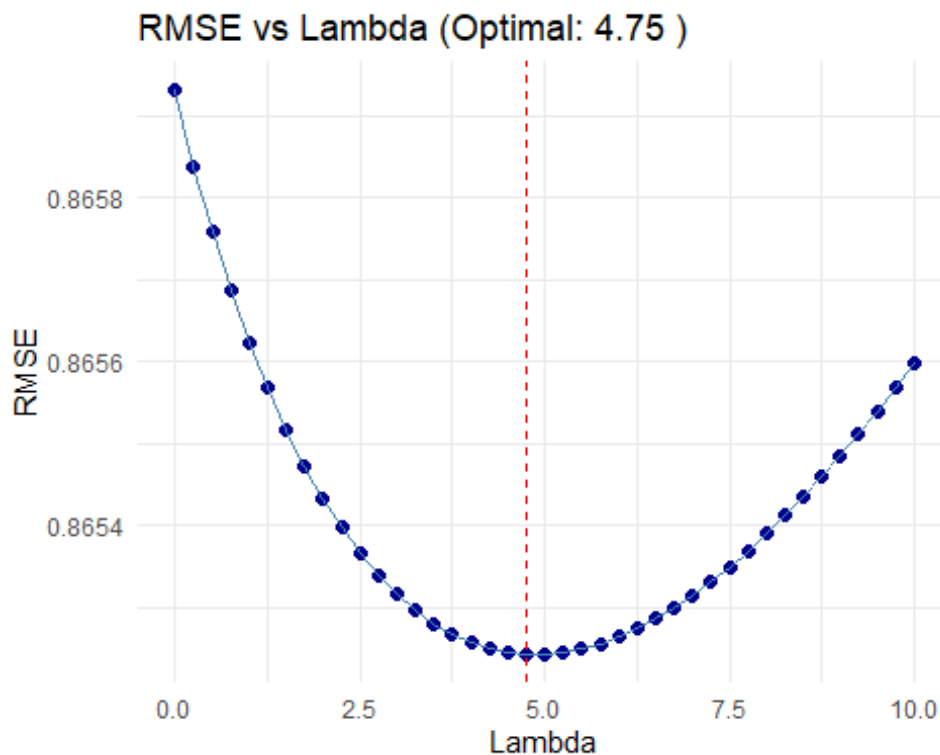
- The **global average model** has an RMSE of 1.05990, which is our baseline.

- Adding the **movie effect** reduces the RMSE to 0.94374, which is a major improvement.
- Adding the **user effect** reduces it further to 0.86593.
- **Regularization** achieves the best performance with an RMSE of 0.86524.

Adding the movie effect explains differences between movies. In addition, incorporating the user effect improves predictions by accounting for users who rate higher or lower than the average, and regularization prevents overfitting.

3.5 Lambda selection

```
tibble(lambda = lambdas, RMSE = rmses) %>%
  ggplot(aes(x = lambda, y = RMSE)) +
  geom_point(size = 2, color = "darkblue") +
  geom_line(color = "steelblue") +
  geom_vline(xintercept = lambda_opt, linetype = "dashed", color = "red") +
  labs(
    title = paste("RMSE vs Lambda (Optimal:", lambda_opt, ")"),
    x = "Lambda",
    y = "RMSE"
  ) +
  theme_minimal()
```



The plot shows how RMSE decreases as lambda increases until the optimal point (4.75). After that, it starts to rise again, which makes sense because too little regularization causes overfitting, while too much causes underfitting.

3.6 3.6 Final result

```
final_result %>%
  knitr::kable(
    caption = "Final Result",
    booktabs = TRUE,
    format.args = list(digits = 5)
  )
```

Final Result

model	lambda	RMSE
Regularized movie + user effect model	4.75	0.86482

The final model achieved an **RMSE of 0.86482** on the test set, which is lower than the 0.86490 target. This shows the importance of incorporating movie and user effects, together with regularization, to build a more accurate model than one based only on the global average.

4 4. Conclusion

4.1 4.1 Summary of the work

In this project, a rating prediction model was developed using regularized movie and user effects.

The final model combined information from millions of ratings to estimate:

- The average quality of each movie.
- Each user's rating tendency.
- The optimal balance between these two factors through regularization.

4.2 4.2 Limitations

Despite its good performance, the model has some limitations:

- It does not consider time information, even though user preferences can change over time.
- It does not use additional movie features such as year, director, or actors.

4.3 4.3 Possible improvements

For future work, some possible improvements would be:

- Incorporate time effects to capture changes in preferences.

- Use additional movie information such as genre, year, and director.
-

5 References

1. GroupLens. (2009). *MovieLens 10M Dataset*.
<https://grouplens.org/datasets/movielens/10m/>
2. HarvardX. (2026). *PH125.9x Data Science: Machine Learning Capstone Project*.
3. Wickham, H., & Grolemund, G. (2017). *R for Data Science*. O'Reilly Media.